Method Chaining

(Also available in CODAP)

Students learn how to chain Methods together, and define more sophisticated subsets.

Lesson Goals	Students will be able to… • Use chaining to write more sophisticated analyses using less code • Identify bugs introduced by chaining methods in the wrong order
Student-facing Lesson Goals	• Let's practice writing functions and combining methods.
Prerequisites	• Simple Data Types • Contracts • Functions: Contracts, Examples & Definitions • Bar and Pie Charts • Solving Word Problems with the Design Recipe • Table Methods
Materials	• Lesson Slides • Animals Starter File • Table Methods Starter File • The Design Recipe: is-dog / is-female (Page 36) • The Design Recipe: is-old / name-has-s (Page 37) • Chaining Methods (Page 38) • Chaining Methods 2: Order Matters (Page 39) • **Classroom visual:** Language Table
Preparation	• All students should log into code.pyret.org (CPO) and open their saved "Animals Starter File". If they don't have the file, they can open a new one from Animals Starter File.

Design Recipe Practice *25 minutes*

Overview

Students practice more of what they learned in the previous lesson, applying the Design Recipe to make table functions that operate on rows of the Animals Dataset. These become the basis of the chaining activity that follows.

Launch

The Design Recipe is a powerful tool for solving problems by writing functions. It's important for this to be like second nature, so let's get some practice using it.

Investigate

- Open your saved Animals Starter File.
- Define the functions on The Design Recipe: is-dog / is-female and The Design Recipe: is-old / name-has-s.

Optional: Combining Booleans

Suppose we want to build a table of Animals that are fixed *and* old, or a table of animals that are cats *or* dogs?

By using the **`and`** and **`or`** operators, we can *combine* Boolean tests, as in: `(1 > 2) and ("a" == "b")`. This is handy for more complex programs! For example, we might want to ask if a character in a video game has run out of health points *and* if they have any more lives. We might want to know if someone's ZIP Code puts them in Texas or New Mexico. When you go out to eat at a restaurant, you might ask what items on the menu have meat and cheese.

For many of the situations where you might use **`and`**, there's actually a much more powerful mechanism you can use, called "Method Chaining"!

Synthesize

- Did you find yourselves getting faster at using the Design Recipe?
- What patterns or shortcuts are you noticing, when you use the Design Recipe?

Chaining *25 minutes*

Overview

Students learn how to compose multiple table operations (sorting, filtering, building) on the same table - a technique called "chaining".

Launch

Suppose we start with some number A, and want to add B, C and D to it. The code below will get the job done:

```
x = A + B                # starting with A, add B
y = x + C                # then add C....
result = y + D           # then add D to get our result
```

- Why is this code ugly, or hard to read?
  - *Many lines of code means more to read and more possible places for bugs*
  - *This code creates names for each step. But we don't really care about* `x` *or* `y` *— we just want the final answer* `result` *!*

We can easily *chain these operators together*, to do all the calculation in one line of code:

```
result = A + B + C + D
```

- Open your saved Table Methods Starter File (or open a new one), and click "Run".
- Can you make a table with a new column called "nametag" that is populated using the `label` function?
- Can you take *that* table, and filter it so it only shows the fixed animals?
- Can you sort *that* table by species?

Let's look at one possible solution to these challenges:

```
x = animals-table.build-column("labels", nametag)    # starting with our table, and add labels
y = with-labels.filter(is-fixed)                     # then filter by is-fixed...
result = fixed-with-labels.order-by("species", true) # then sort by species to get our result
```

- Why is this code ugly, or hard to read?
  - *Many lines of code means more to read and more possible places for bugs*
  - *This code creates names for each step. But we don't really care about* `x` *or* `y` *— we just want the final answer* `result` *!*

Pyret allows table methods to be *chained together*, so that we can build, filter *and* order a Table in one shot. For example:

```
result = animals-table.build-column("labels", nametag).filter(is-fixed).order-by("species", true)
```

Let's walk through this line of code one step at a time:

- We take the `animals-table`, and produce a new table with an extra column called `label`.
- Then call *that* Table's `.filter` method, producing a new table with a `label` column and only rows for fixed animals.
- Then we call *that* Table's `order-by` method, producing a new, sorted table of fixed animals with a `label` column.

Teaching Tip

Use different color markers to draw *nested boxes* around each part of the expression, showing each of the three steps described above.

It can be difficult to read code that has lots of method calls chained together, so we can add a line-break before each `.` to make it more readable. Here's the exact same code, written with each method on its own line:

```
# get a table with the nametags of all
# the fixed animals, order by species
animals-table
  .build-column("label", nametag)
  .filter(is-fixed)
  .order-by("species", true)
```

Order matters: Build , Filter, Sort.

Suppose we want to build a column and then use it to filter our table. If we use the methods in the wrong order (trying to filter by a column that doesn't exist yet), we might wind up crashing the program. Even worse, the program might work, but produce results that are incorrect!

- Take a minute to think about what code you would write to sort the animals table by the kilograms column.
- Do you think `animals-table.order-by("kilograms", true).build-column("kilograms", kilos)` will generate the table we want? Why or why not? Test your hypothesis by typing the code that you think will build the table into the starter file!

Investigate

- How well do you know your table methods ?
- Complete Chaining Methods and Chaining Methods 2: Order Matters in to find out.

Synthesize

Ask students about their answers to Chaining Methods 2: Order Matters.

- Which ones produced an error?
- Why will they produce an error?
- How could they be fixed?

Can students trigger a similar error in their Table Methods starter file?
As our analysis gets more complex, chaining is a great way to re-use code we've already written. And less code means a smaller chance of bugs.